

RobuQ: Pushing DiTs to W1.58A2 via Robust Activation Quantization

Kaicheng Yang^{1*} Xun Zhang^{1*} Haotong Qin² Yucheng Lin¹ Kaisen Yang³ Xianglong Yan¹ Yulun Zhang^{1†}

Abstract

Diffusion Transformers (DiTs) have emerged as a powerful backbone for image generation, offering superior scalability over U-Nets. However, their practical deployment is hindered by significant computational costs. While Quantization-Aware Training (QAT) shows promise, its application to DiTs is challenged by the high sensitivity and complex distributions of activations. Identifying activation quantization as the primary bottleneck for low-bit settings, we propose **RobuQ**, a systematic QAT framework. We first establish a strong ternary weight (W1.58A4) baseline. Building on this, we introduce **RobustQuantizer**, which utilizes the Hadamard transform to convert unknown per-token distributions into normal distributions. Furthermore, we propose **AMPN**, the first **Activation-only Mixed-Precision Network** pipeline, applying ternary weights globally while allocating layer-specific activation precisions to eliminate information bottlenecks. Extensive experiments demonstrate that **RobuQ** achieves state-of-the-art performance on **ImageNet-1K**, representing the first stable image generation with activations quantized to an average of 2 bits. Code is available at <https://github.com/racoonykc/RobuQ>.

1. Introduction

Recent advances in quantization-aware training (QAT) have revealed a fundamental asymmetry between weight and activation quantization in deep neural networks (Zheng et al., 2025; Feng et al., 2025a; Wang et al., 2025b; He et al., 2024b). In particular, diffusion transformer (DiT) models (Peebles & Xie, 2023), which have demonstrated strong performance in generative tasks, present unique challenges for efficient quantization due to their deep archi-

^{*}Equal contribution ¹Shanghai Jiao Tong University, Shanghai, China ²ETH Zürich, Zurich, Switzerland ³Tsinghua University, Beijing, China. Correspondence to: Yulun Zhang[†] <yulun100@gmail.com>.

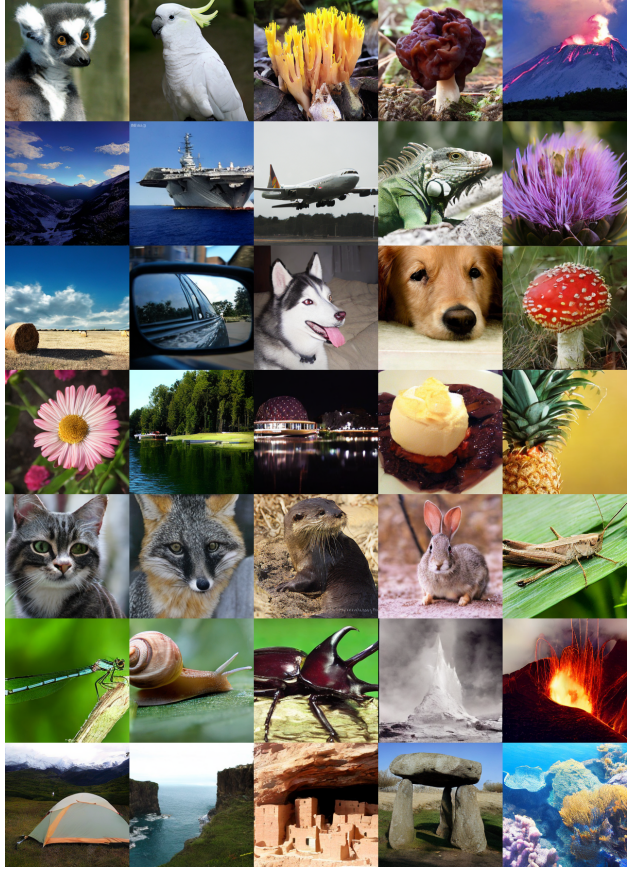


Figure 1. **RobuQ enables DiTs to generate competitive results at ultra-low bit setting.** We select 256×256 images from W1.58A3 quantized DiT-XL/2 trained on ImageNet-1K.

tectures and the complex distribution of activation values. While prior studies have shown that ternary quantization of weights can achieve nearly lossless accuracy (Ma et al., 2024), activation quantization remains substantially more difficult—especially for large-scale datasets like ImageNet-1K (Russakovsky et al., 2015), where the lowest reported activation bit-width is still 4 bits (Feng et al., 2025b). This highlights an opportunity to further reduce activation precision in DiT models without sacrificing generative quality greatly.

In this work, we focus on the quantization of DiT models and conduct a systematic analysis to identify activation quantization as the principal challenge in ultra-low bit settings. Building on this observation, we first establish a strong W1.58A4 DiT quantization baseline. We then theo-

retically demonstrate that, under our modeling assumptions, the Hadamard transform can consistently project diverse and irregular activation distributions in DiT into a standard normal form. Exploiting this property, we propose the RobustQuantizer including the construction of an advanced W1.58A4 baseline, the Hadamard transform and the robust per-token Gauss quantizer, thereby enabling highly efficient and distribution-agnostic quantization in both uniform and non-uniform quantization.

Mixed-precision quantization has recently emerged as a promising strategy to overcome the limitations of uniform ultra-low-bit quantization (Feng et al., 2025a; Zhao et al., 2024a; Kim et al., 2025; Feng et al., 2025b). We introduce the first activation-only mixed-precision quantization network (AMPN) for DiT, and explore activation bit-width allocation strategies within this framework at ultra-low bit setting. Using AMPN, we achieve SOTA image generation on ImageNet at an ultra-low precision of W1.58A3 (as seen in Fig. 1), while maintaining stable performance without collapse at the even lower bit-width of W1.58A2. Extensive experiments on both unconditional and conditional generation tasks demonstrate our method’s superior performance over SOTA techniques.

Our main contributions are summarized as follows:

- Through comprehensive study, we identify activation quantization as the central bottleneck for DiTs to achieve ultra-low bit quantization. Building upon recent work, we establish a strong **baseline** for ternary weight quantization with a W1.58A4 DiT model, achieved through the integration of an SVD-initialized low-rank branch and Hadamard transform.
- We first **theoretically** demonstrate that the widely used Hadamard transform, under our modeling assumptions, can convert arbitrary activation distributions in DiT models to a per-token normal distribution. Leveraging this property, our **RobustQuantizer** supports both uniform and non-uniform quantization, achieving SOTA performance on W1.58A4.
- We introduce **AMPN**, the first DiT quantization scheme to focus exclusively on activation-only mixed-precision, and conduct a thorough exploration of activation bit-width allocation. Our method achieves SOTA performance at W1.58A3 and, furthermore, maintains stable training without collapse at an ultra-low bit setting of W1.58A2.
- Extensive evaluations across unconditional generation and conditional generation with DiT demonstrate that our quantization framework **RobuQ**, including RobustQuantizer and AMPN, consistently surpasses previous SOTA methods in both efficiency and performance, significantly advancing the feasibility of DiTs under resource constraints.

2. Related Works

2.1. Diffusion Transformers

Diffusion Models (DMs) have demonstrated impressive generative capabilities across a wide range of tasks (Chen et al., 2020; Hu et al., 2022; Rombach et al., 2022; Chen et al., 2023; He et al., 2023; Li et al., 2023b;a; Liu et al., 2024; Li et al., 2024; He et al., 2024a; Ho et al., 2020; Zhao et al., 2024b; Peebles & Xie, 2023). Recent research has focused on replacing the conventional U-Net (Ronneberger et al., 2015) backbone with Transformer-based (Vaswani et al., 2017) architectures to build more powerful generative models (Croitoru et al., 2023; Rombach et al., 2022; Yang et al., 2023). Among these, Diffusion Transformers (DiTs) (Peebles & Xie, 2023) has achieved remarkable performance in image generation, exhibiting strong scalability and significant potential for broader applications. Despite its exceptional performance, DiT still demands substantial computational resources, including high memory usage and processing power, to generate high-quality images, which significantly hinders its applicability in resource-constrained scenarios.

2.2. Quantization

Quantization techniques (K Esser et al., 2019; Lv et al., 2024; Zhang et al., 2024; Zhou et al., 2016) compress and accelerate neural networks by reducing the numerical precision of weights and activations (e.g., from 32-bit floating-point to 1–8-bit integers). However, applying quantization to generative tasks presents unique challenges due to the dynamic temporal nature of the diffusion process and the complex spatial structures involved (Chen et al., 2024; He et al., 2024b).

To further improve the efficiency of neural network quantization, recent research has explored even lower bit-width regimes, such as ternarization (three-value quantization) (Lu et al., 2024; Ma et al., 2024; Wang et al., 2025a) and extreme low-bit quantization (e.g., 1-bit, 2-bit) (Zheng et al., 2024; 2025; Zhao et al., 2026b;a). These approaches significantly reduce both memory footprint and computational complexity, but they typically struggle to maintain sufficient model expressiveness and high generation quality, especially in the context of large generative models that require intricate representations.

To address information loss caused by aggressive quantization, orthogonal transformations and decomposition-based compensation have been introduced into quantization pipelines (Hu et al., 2025; Lin et al., 2025; Ashkboos et al., 2024; Liu et al., 2025b; Xu et al., 2026). By decorrelating weights or activations before quantization (e.g., via SVD, Hadamard, or other orthogonal transforms), these methods redistribute quantization errors and better preserve information, enabling more accurate low-bit quantization for generative models.

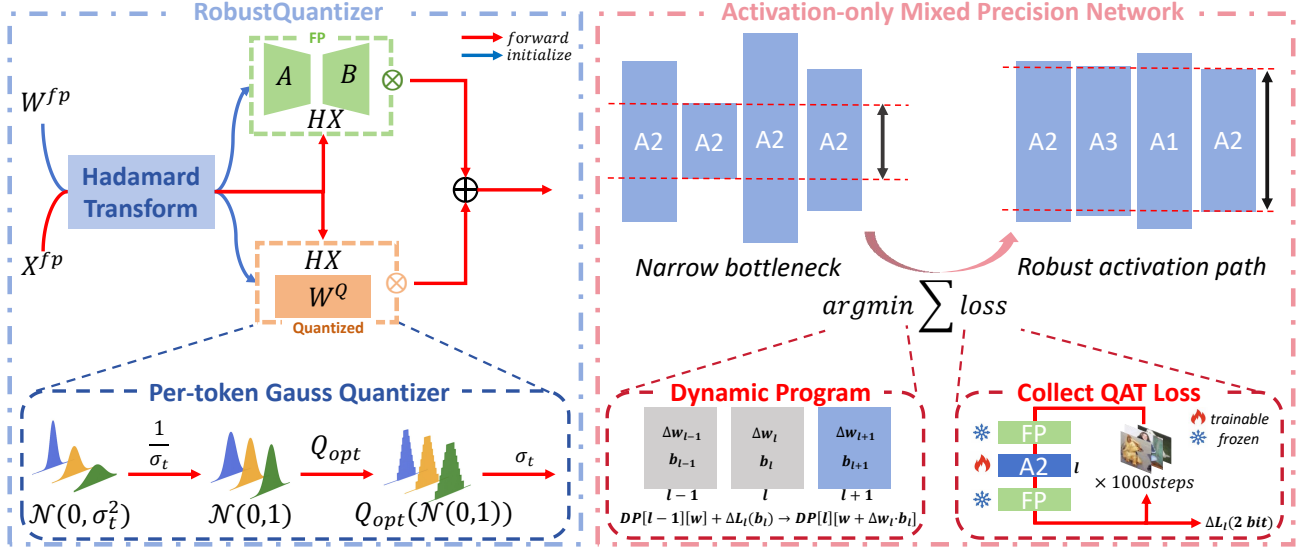


Figure 2. Overall Framework of Our Quantization Pipeline.

Moreover, mixed-precision quantization has emerged as an effective strategy to balance efficiency and performance (Feng et al., 2025a; Zhao et al., 2024a; Kim et al., 2025; Feng et al., 2025b). Instead of assigning a uniform bit-width to all layers or modules, mixed-precision methods allocate higher precision to sensitive components and lower precision elsewhere, either through heuristic rules or data-driven optimization. This technique enhances quantization robustness and overall performance.

3. Method

3.1. Analysis

Empirical evidence indicates that DiT models (Peebles & Xie, 2023) exhibit inferior performance in the low bit-width regime compared to U-Net-based (Ronneberger et al., 2015) LDM models (Rombach et al., 2022). Currently, DiT quantization is often limited to a W4A4 configuration (Liu & Zhang, 2024; Wu et al., 2024; Hwang et al., 2025; Chen et al., 2025), whereas LDM-class models have advanced to W1A4 and even W1A1 precedents (Zheng et al., 2024; 2025). This significant gap motivates a thorough investigation into activation quantization for DiTs. We identify three key challenges that hinder effective low-bit-width activation quantization in DiT models:

- **Issue 1: Lack of QAT Exploration for Ultra-low-bit Configurations.** Existing methods have primarily focused on Post-Training Quantization (PTQ) (He et al., 2024b; Wang et al., 2025b), without a thorough investigation into the boundaries of activation bit-width under the Quantization-Aware Training (QAT) framework. Compared with PTQ, QAT can explicitly optimize model parameters during training to compensate for quantization errors, thereby offering a more promising and effective route to stable ultra-low-bit deployment.
- **Issue 2: Diverse and Complex Activation Distributions.** Unlike other architectures, DiTs exhibit highly

varied activation distributions across different layers and tokens (Zhao et al., 2025a), posing a significant challenge due to the lack of a unified quantizer.

- **Issue 3: Potential Activation Bit-width Bottlenecks.** We find the existence of specific layers within DiT models that are particularly sensitive to activation bit-width compression, which fundamentally prevents further quantization to lower activation bit-widths.

Based on the above issues, it becomes necessary to conduct a dedicated study on ultra-low-bit activation quantization for DiT models. Our goal is to address the unique distributional and architectural challenges of DiT, and to develop strategies that maximize compression while preserving generative fidelity. Such targeted investigation is essential not only for reducing deployment costs but also for pushing the practical boundaries of DiT quantization into regimes previously thought unattainable.

3.2. RobustQuantizer

3.2.1. BASELINE

Building upon the successful W1.58A4 configuration of BitNetv2 (Wang et al., 2025a), we establish it as our initial baseline. Specifically, we apply a Hadamard transformation (Yarlagadda & Hershey, 1993), a type of orthogonal transformation to the *proj* and *fc2* layers within the DiT modules. The Hadamard transformation is applied to both the weights and activations, where $W \leftarrow HW$ and $X \leftarrow HX$. Here, the Hadamard matrix of order n is defined recursively:

$$H_1 = (1), \quad H_{2^n} = \frac{1}{\sqrt{2}} \begin{pmatrix} H_n & H_n \\ H_n & -H_n \end{pmatrix} \quad (n \geq 1). \quad (1)$$

We adopt two distinct strategies for weight and activation quantization. For weight quantization $Q_w(\cdot)$, we use a channel-wise ternarization quantizer based on the principles of BitNetV2. This maps the FP weights W to discrete

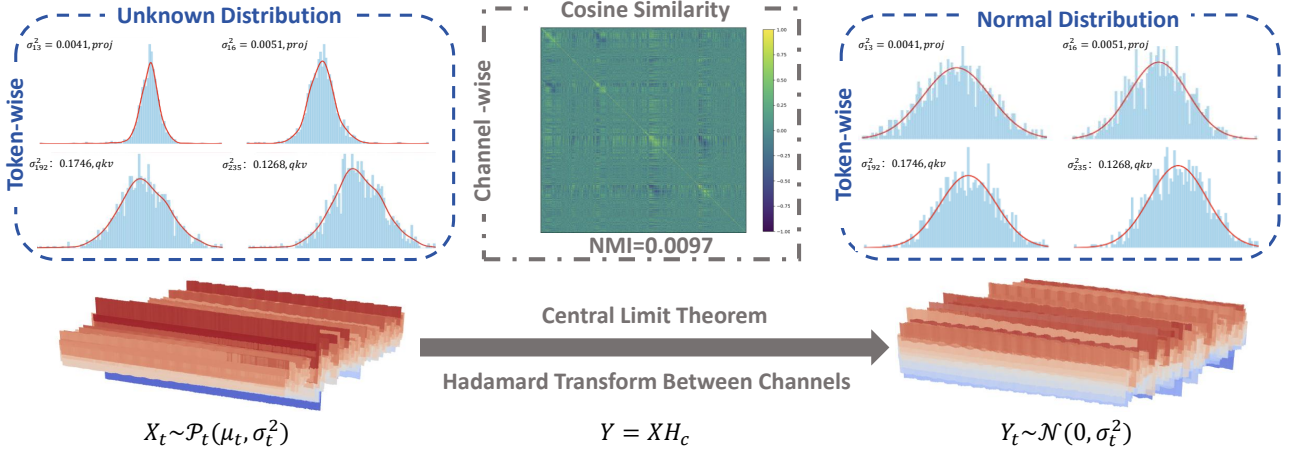


Figure 3. Illustration of how the Hadamard transforms per-token unknown distributions (left) into a known per-token normal distribution (right). Average NMI is computed across different channels.

values per channel, as shown in the following equation:

$$Q_w(W) = \alpha \cdot \text{RoundClip}\left(\frac{W}{\gamma + \epsilon}, -1, 1\right), \quad (2)$$

where $\alpha = \text{mean}(|W|)$, $\gamma = \frac{1}{mn} \sum_{i,j} |W_{ij}|$, and ϵ is a small constant to avoid division by zero. The RoundClip function is defined as $\text{RoundClip}(x, a, b) = \min(\max(\text{round}(x), a), b)$. For activation quantization, we employ a straightforward per-token min-max quantization strategy to determine the scaling range. The quantized value $Q_x(\mathbf{x})$ for an activation tensor $\mathbf{x}$ is computed as:

$$Q_x(\mathbf{x}) = \text{clamp}\left(\left\lfloor \frac{\mathbf{x}}{\delta} \right\rfloor + \lambda, 0, 2^b - 1\right), \quad (3)$$

where $\delta = \frac{\max(\mathbf{x}) - \min(\mathbf{x})}{2^b - 1}$ is the scaling factor, b is the bit-width, $\lfloor \cdot \rfloor$ denotes the floor operation, and $\lambda = -\left\lfloor \frac{\min(\mathbf{x})}{\delta} \right\rfloor$ is the zero-point that enables asymmetric quantization.

3.2.2. ENHANCED BASELINE WITH INTEGRATED TECHNIQUES

Next, we turn our attention to other state-of-the-art methods. By drawing on techniques from SVD-Quant (Li et al., 2025) and BiMaCoSR (Liu et al., 2025a), we introduce a SVD-initialized low-rank matrix branch for compensation, which operates in FP. As illustrated in Fig. 2 (left), the initialization process begins with the FP weights W . First, a Hadamard transform is applied to W . Then, a truncated SVD is performed on the transformed matrix to construct the low-rank approximation, which is subsequently factorized into A and B . The decomposition is as follows:

$$WH \approx AB = U_r \Sigma_r V_r^T. \quad (4)$$

Here, H denotes the Hadamard matrix. The matrices U_r , Σ_r , and V_r are obtained by retaining the top $r = 16$ dominant singular values and their corresponding singular vectors. The main quantized weight matrix W^Q in the lower branch

is also derived from the transformed weights WH :

$$W^Q = Q_w(WH - AB) = Q_w(W_{\text{res}}). \quad (5)$$

The original weight W is then approximated as follows:

$$W = (AB + W_{\text{res}})H^T \approx ABH^T + Q_w(W_{\text{res}})H^T. \quad (6)$$

During the forward pass, the input X is passed through a Hadamard transform and then into the Q_G , as shown in Eq. 8. Q_G refers to the Per-token Gauss Quantizer introduced in Section 3.2.4. The final output is the sum of the outputs from the FP low-rank branch and the main quantized branch. Although BitNetv2 restricted the Hadamard transform to *proj* and *fc2* layers, extending it to all layers, as we do here, stabilizes activation distributions and mitigates residual imbalances.

3.2.3. HADAMARD TRANSFORM CREATES A PER-TOKEN NORMAL DISTRIBUTION

We argue that the Hadamard transform provides more than simple activation smoothing (Kolb et al., 2023): it converts per-token activations from arbitrary distributions into predictable, approximately normal ones. This property, visualized in Fig. 3, motivates our **RobustQuantizer**.

Formally, consider the input $X \in \mathbb{R}^{T \times C}$. We have observed the following three properties:

(i) **Token-wise:** Activations across tokens within a layer share a distribution shape but differ in mean and variance, and these distributions vary significantly across layers, leading to quantization errors.

(ii) **Channel-wise:** Channels are nearly independent, with low normalized mutual information (NMI), which is a key property to satisfy the CLT assumptions (Gnedenko & Kolmogorov, 1954).

(iii) **Hadamard Matrix Property:** The normalized

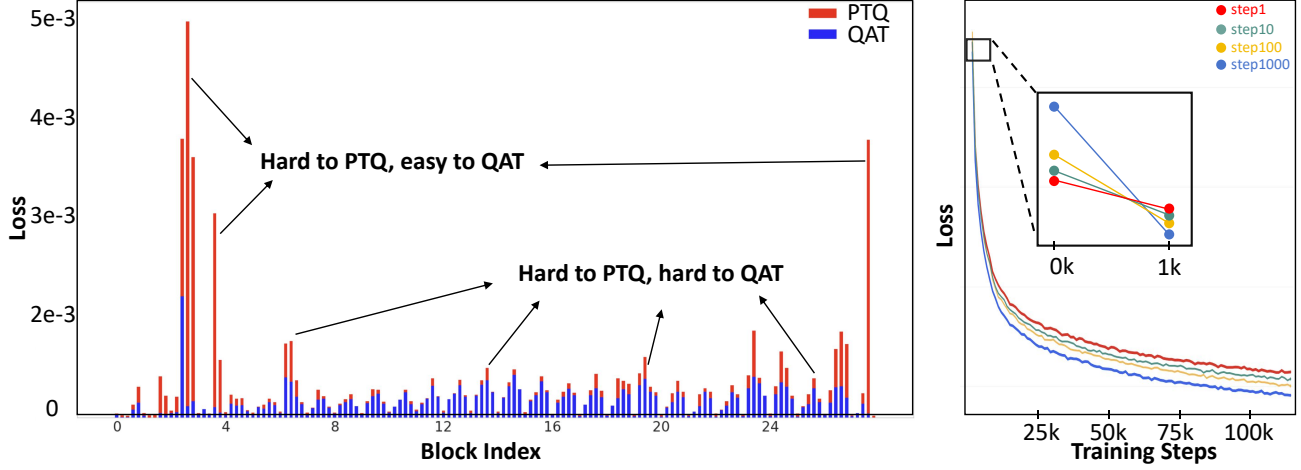


Figure 4. An illustration of why PTQ sensitivity metrics fail for ultra-low-bit QAT mixed-precision. **Left:** Visualization of accuracy loss for different linear layers with W1.58A2 quantization under PTQ and QAT (1,000 training steps). **Right:** Mixed-precision configurations derived from more QAT steps achieve a worse initial loss but a better final convergence loss.

Hadamard matrix H_C has entries of $\pm 1/\sqrt{C}$, which ensures an equal variance across the resulting transformed channels in one token. Thus, per-token activations $X_t = (X_{t,1}, \dots, X_{t,C})$, with $X_{t,c} \sim \mathcal{P}_{t,c}(\mu_{t,c}, \sigma_{t,c}^2)$, become

$$\text{Var}(Y_{t,c}) = \frac{1}{C} \sum_{j=1}^C \sigma_{t,j}^2 \triangleq \sigma_t^2. \quad (7)$$

By the Generalized CLT, Y_t converges to $\mathcal{N}(0, \sigma_t^2)$, i.e., an identically distributed Gaussian for each token. This insight provides a principled theoretical foundation for achieving robust and effective per-token quantization. Further information can be found in Appendix A.

3.2.4. FROM HADAMARD NORMALIZATION TO PER-TOKEN GAUSS QUANTIZATION

Building upon our prior analysis of how the Hadamard transform produces a per-token normal distribution, we now design the **Per-token Gauss Quantizer** $Q_G(\cdot)$ to maximally leverage this property. We present two versions of our quantizer, a **uniform** and a **non-uniform** variant. The complete process involves per token normalization using dynamically computed mean and variance and quantization with a pre-computed optimal quantizer as shown in Fig. 2 (left lower). We obtain this optimal quantizer, denoted as Q_{opt} , by using the Lloyd-Max algorithm. The complete quantization and dequantization process for an activation vector x can be expressed as:

$$x \approx Q_G(x) = \sigma_t \cdot H^T \cdot Q_{\text{opt}}\left(\frac{Hx}{\sigma_t}\right). \quad (8)$$

Therefore, the forward of the quantized and low-rank FP branches can be expressed as:

$$Wx \approx \underbrace{ABHx}_{\text{FP}} + \underbrace{Q_w(W_{\text{res}}) \cdot Q_{\text{opt}}\left(\frac{Hx}{\sigma_t}\right) \cdot \sigma_t}_{\text{quantized}}. \quad (9)$$

3.3. Activation-only Mixed-Precision Network

3.3.1. NAIVE PIPELINE DESIGN

We design a simple activation-only mixed-precision network (AMPN) pipeline to alleviate bottlenecks caused by uniform bit-width quantization, as shown in Fig. 2 (right). All weights are fixed to ternary (W1.58), while each activation layer $\ell \in \{1, \dots, L\}$ selects a bit-width $b_\ell \in \mathcal{B} = \{1, 2, 3, 4\}$. The goal is to minimize accuracy loss under a target average activation bit-width $\bar{B}_{\text{tgt}}$.

To build a layer-wise sensitivity profile, we randomly sample 1,000 validation examples across timesteps and compute the mean loss gap $\Delta L_\ell(b_\ell)$ between the quantized and full-precision models at bit-width b . This metric enables a fast estimation of per-layer degradation. We then formulate bit allocation as a Dynamic Programming (DP) problem, where the objective is to minimize total loss under a resource budget. Here, w_ℓ is the layer-wise weight that adjusts the bitwidth contribution of each layer according to its FLOPs proportion in DiT-Block (e.g., the w_ℓ of mlp.fc1 is 1.334).

Among these layers, certain components are fixed for stability: the attention scores are quantized to 8 bits, and the adaLN layer to 4 bits, due to their high sensitivity yet negligible FLOPs cost (together accounting for about 2–3% of the total block computation). The optimization can be written as

$$\min_{\{b_\ell \in \mathcal{B}\}} \sum_{\ell=1}^L \Delta L_\ell(b_\ell) \quad \text{s.t.} \quad \frac{1}{W_{\text{tot}}} \sum_{\ell=1}^L w_\ell b_\ell \leq \bar{B}_{\text{tgt}}, \quad (10)$$

where $W_{\text{tot}} = \sum_{\ell=1}^L w_\ell$ is the total FLOPs. We solve this with DP. Let $\text{DP}[\ell][w]$ be the minimal cumulative loss after assigning bits to the first ℓ layers with accumulated weighted cost w . For the purpose of discretization, each individual

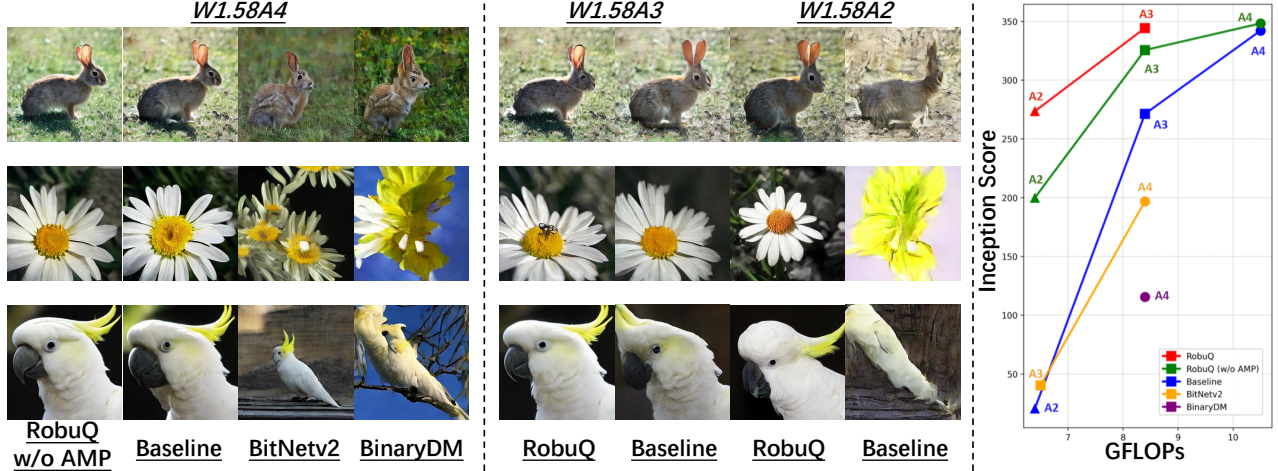


Figure 5. Visualization of the performance and efficiency of RobuQ and comparative approaches. **Left:** Our proposed RobuQ and baseline significantly outperform previous methods on W1.58A4. **Middle:** RobuQ maintains stable generation under A3 and A2 compared to collapsed baseline. **Right:** The RobuQ series achieve higher Inception Scores under the same FLOPs.

layer’s FLOPs and the target budget can be written as

$$\Delta w_\ell = \left\lfloor \beta \frac{w_\ell}{W_{\text{tot}}^{\text{dp}}} \right\rfloor, \quad B = \left\lfloor \beta \bar{B}_{\text{tgt}} \right\rfloor, \quad (11)$$

where β is a resolution factor controlling granularity (e.g. $\beta=1,000$) and $W_{\text{tot}}^{\text{dp}} = \sum_{\ell \in \mathcal{L}_{\text{dp}}} w_\ell$ is the FLOPs of layers optimized by DP. The recurrence relation is formally defined as follows,

$$\text{DP}[\ell][w + \Delta w_\ell \cdot b_\ell] = \min_{b_\ell \in B} \left\{ \begin{array}{l} \text{DP}[\ell][w + \Delta w_\ell \cdot b_\ell], \\ \text{DP}[\ell - 1][w] + \Delta L_\ell(b_\ell) \end{array} \right\}, \quad (12)$$

We initialize $\text{DP}[0][0] = 0$ and $\text{DP}[0][w > 0] = +\infty$. The optimal cost can be formulated as

$$w^* = \arg \min_{0 \leq w \leq B} \text{DP}[L][w], \quad (13)$$

from which the optimal allocation $\{b_\ell^*\}$ is recovered by backtracking through the solution space.

3.3.2. ULTRA-LOW-BIT QAT

Mixed-precision methods (Feng et al., 2025a; Zhao et al., 2024a; Kim et al., 2025; Feng et al., 2025b) have traditionally employed PTQ to collect parameters, as they are often applied in mid-bit configurations. However, our work targets ultra-low-bit quantization under the QAT framework. In this setting, even if a layer exhibits large quantization errors during PTQ, the model can still compensate for these errors during training, making QAT more adaptable, as shown in Fig. 4 (left). On the other hand, a low quantization error observed in PTQ does not necessarily ensure a consistent or further reduction in quantization errors during the subsequent QAT process.

To investigate this, we explored training the quantized layers for different numbers of steps while collecting quantization

errors. Specifically, we trained for 1, 10, 100, and 1,000 steps, using the same learning rate as standard training. Our findings revealed that while schemes with fewer training steps (such as 1 and 10 steps) initially exhibited lower quantization errors, those trained with more steps (such as 1,000 steps) achieved a significantly lower final convergence loss, as shown in Fig. 4 (right). This aligns with our hypothesis that additional QAT steps allow the model to better adjust to the quantization process, gradually improving its performance and robustness over time.

4. Experiments

4.1. Setup

Datasets and Evaluation Metrics. We evaluate pre-trained class-conditional DiT-XL/2 models at 256×256 resolution on ImageNet-1K (Russakovsky et al., 2015) and FFHQ (Karras et al., 2019). The DDPM solver (Ho et al., 2020) with 250 sampling steps is employed for the generation process. For all methods under evaluation, we uniformly sample a total of 10,000 generated images for both the ImageNet-1K 256×256 and FFHQ 256×256 benchmarks. We use four metrics to assess generated image quality: Fréchet Inception Distance (FID) (Heusel et al., 2017), spatial FID (sFID) (Salimans et al., 2016; Nash et al., 2021), Inception Score (IS) (Salimans et al., 2016; Barratt & Sharma, 2018), and Precision, all computed using the ADM toolkit (Dhariwal & Nichol, 2021).

Compared Methods. We primarily benchmark our RobuQ series against state-of-the-art ultra-low-bit QAT approaches, including BitNetv2 (Wang et al., 2025a) and BinaryDM (Zheng et al., 2025), alongside our constructed strong baselines (Ashkboos et al., 2024; Li et al., 2025). To ensure a fair comparison, we align the training configurations across all QAT methods. In addition, to further investigate the intrinsic robustness of our proposed quantizer, we also provide a brief analysis under the calibration-free W4A4 PTQ

Table 1. Performance on ImageNet-1K 256×256 and FFHQ 256×256 under different settings.

Setting	Method	Bit-width (W/A)	IS $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$
ImageNet steps= 50 cfg = 1.5	FP	32/32	239.50	6.62	21.10	0.7849
	BinaryDM †	1.58/4	25.63	62.91	38.28	0.3765
	Bitnetv2	1.58/4	44.32	41.59	34.09	0.5002
	Baseline	1.58/4	95.07	20.82	27.53	0.6152
	RobuQ (w/o AMP)	1.58/4	103.24	17.97	26.95	0.6577
	Baseline	1.58/3	51.31	40.23	35.64	0.4946
	RobuQ (w/o AMP)	1.58/3	83.84	24.44	29.18	0.6001
	RobuQ	1.58/3	93.75	21.40	26.99	0.6190
	Baseline	1.58/2	10.63	120.49	62.29	0.2091
	RobuQ (w/o AMP)	1.58/2	45.65	43.31	38.89	0.4917
	RobuQ	1.58/2	66.74	30.30	30.66	0.5680
ImageNet steps= 50 cfg = 4.0	FP	32/32	478.35	19.11	21.61	0.9298
	BinaryDM †	1.58/4	115.52	17.08	23.15	0.7230
	Bitnetv2	1.58/4	196.78	11.69	21.44	0.8370
	Baseline	1.58/4	342.07	12.82	20.05	0.9092
	RobuQ (w/o AMP)	1.58/4	349.22	12.64	19.69	0.9186
	Baseline	1.58/3	254.92	10.83	21.68	0.8585
	RobuQ (w/o AMP)	1.58/3	325.56	12.31	19.94	0.9053
	RobuQ	1.58/3	342.94	12.71	19.87	0.9129
	Baseline	1.58/2	20.65	75.02	35.81	0.2812
	RobuQ (w/o AMP)	1.58/2	200.00	11.97	21.73	0.8188
	RobuQ	1.58/2	273.58	11.06	21.57	0.8751
ImageNet steps= 150 cfg = 4.0	FP	32/32	479.72	19.67	22.94	0.9301
	BinaryDM †	1.58/4	124.78	15.50	21.66	0.7575
	Bitnetv2	1.58/4	206.28	11.72	20.62	0.8591
	Baseline	1.58/4	344.43	13.92	20.75	0.9167
	RobuQ (w/o AMP)	1.58/4	348.40	13.82	20.32	0.9225
	Baseline	1.58/3	271.27	11.55	20.76	0.8876
	RobuQ (w/o AMP)	1.58/3	333.62	13.65	20.75	0.9180
	RobuQ	1.58/3	342.73	14.27	20.63	0.9247
	Baseline	1.58/2	23.22	68.04	30.84	0.2999
	RobuQ (w/o AMP)	1.58/2	220.47	11.17	20.10	0.8573
	RobuQ	1.58/2	281.73	11.86	21.94	0.8922
FFHQ steps= 50 Uncondition	FP	32/32	N/A	11.71	28.88	0.7526
	Bitnetv2	1.58/4	N/A	66.55	64.49	0.3499
	Baseline	1.58/4	N/A	34.32	44.37	0.5771
	RobuQ (w/o AMP)	1.58/4	N/A	25.62	37.15	0.6228
	Baseline	1.58/3	N/A	35.11	43.07	0.5988
	RobuQ	1.58/3	N/A	28.11	38.43	0.6128
	Baseline	1.58/2	N/A	59.50	62.58	0.4159
	RobuQ	1.58/2	N/A	38.13	42.29	0.5568

† For fairness, we swapped BinaryDM’s binarization for ternarization.

setting, comparing it with SoTA methods like QueST (Wang et al., 2025b), PTQ4DiT (Wu et al., 2024), Q-DiT (Chen et al., 2025) and SVDQuant (Li et al., 2025).

Training and Quantization Details. All experiments are conducted with PyTorch (Paszke et al., 2019) on a single NVIDIA RTX A6000-48GB GPU. For all QAT methods, we use the AdamW optimizer (Loshchilov & Hutter, 2019) (learning rate= 10^{-5} , weight decay=0) with a batch size of 8 and train for 350k iterations. We keep the embedding and final layer in full precision across all methods, and maintain 8-bit precision for activation-activation matrix multiplication operations, as they constitute a small fraction of the computation and exhibit high sensitivity to quantization. This single-GPU setting is used for fair comparison across methods, while Sec. 4.7 further reports an 8-GPU QAT study to probe the performance under a larger training budget.

4.2. Main Result

As presented in Table 1, our RobuQ demonstrates comprehensive superiority in the ultra-low-bit regime. On ImageNet-1K 256×256 (cfg=1.5) and FFHQ (unconditional), RobuQ consistently outperforms strong baselines and competing methods like BinaryDM and BitNetv2 across all bit-width configurations.

Notably, our method achieves exceptional performance even at extremely low bit-widths: our **W1.58A2** model achieves an FID of 30.30, which is significantly better than BinaryDM (FID 62.91) and BitNetv2 (FID 41.59) operating at higher precision (W1.58A4). This result strongly validates that our RobustQuantizer effectively mitigates the quantization noise in low-bit diffusion models. When employing a higher guidance scale (cfg=4.0), our method maintains the best metric scores, although we observe that FID exhibits an

RobuQ: Pushing DiTs to W1.58A2 via Robust Activation Quantization

Table 2. Ablation studies on ImageNet-1K 256×256 . Timesteps are 50 and cfg is 1.5.

Method	IS $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$	Method	IS $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$
BitNetv2	44.32	41.59	34.09	0.5002	Baseline	95.07	20.82	27.53	0.6152
+ LRB	68.82	29.59	31.35	0.5807	+Non-uniform Quantizer	96.19	20.33	27.52	0.6262
+ LRB + All Hadamard	95.07	20.82	27.53	0.6152	+Uniform Quantizer	103.24	17.97	26.95	0.6577
(a) Baseline construction at W1.58A4.					(b) Per-token Gauss Quantizer at W1.58A4.				
QAT-step	Method	IS $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$	Training-time $\downarrow$			
N/A	RobustQuantizer	45.65	43.31	38.89	0.4917	126.0h			
1	+AMP	50.67	41.06	34.58	0.5028	3.1h+126.0h			
10		52.12	39.01	32.50	0.5092	3.7h+126.0h			
100		56.31	37.45	32.61	0.5097	9.5h+126.0h			
500		57.93	36.57	33.46	0.5213	36.0h+126.0h			
1,000		66.74	30.30	30.66	0.5680	78.5h+126.0h			
1,500		66.23	30.56	30.23	0.5701	121.0h+126.0h			

(c) AMPN at W1.58A2. Training-time comprises metric collection and actual training.

Table 3. Inference efficiency of our proposed RobuQ of DiT-XL/2 on ImageNet-1K 256×256 .

cfg=4.0 step=50	FP	QueST	BinaryDM	Bitnetv2	Baseline	RobuQ	RobuQ	RobuQ
W/A	32/32	4/4	1.58/4	1.58/4	1.58/4	1.58/4	1.58/3	1.58/2
Size (MB) $\downarrow$	2,575.4	341.22	148.13	148.13	194.75	194.75	194.75	194.75
FLOPs (G) $\downarrow$	114.52	14.94	8.04	8.04	10.07	10.07	8.34	6.61
IS $\uparrow$ / Precision $\uparrow$	478.35/0.9298	42.07/0.2651	115.52/0.7230	196.78/0.8370	342.07/0.9092	349.22/0.9186	344.29/0.9083	273.58/0.8751

atypical trend across all quantized methods compared to FP baselines. Overall, RobuQ establishes a new state-of-the-art for ternary weight quantization.

4.3. Ablation Study

Baseline Construction. Table 2a validates our baseline design. Introducing a low-rank branch (LRB) to BitNetv2 reduces FID from 41.59 to 29.59. Applying Hadamard transformation to all linear layers further lowers FID to 20.82, confirming the necessity of these components for stable low-bit training.

Per-token Gauss Quantizer. As shown in Table 2b, while the non-uniform quantizer offers marginal gains over the baseline, the uniform quantizer significantly boosts IS to 103.24 and lowers FID to 17.97. Its superior robustness to approximation errors and ease of deployment make it the optimal choice for our framework; we provide a deeper theoretical comparison and additional experiments on larger models in Appendix D.2.

QAT Steps in AMP. Table 2c analyzes the impact of metric collection steps. FID improves consistently with more steps, plateauing around 1,000 steps (FID 30.30). Since increasing to 1,500 steps yields negligible gains at a significant computational cost, we adopt 1,000 steps as the best efficiency-accuracy trade-off.

4.4. Efficiency Analysis

Table 3 demonstrates that RobuQ achieves the best efficiency-accuracy trade-off. Notably, our RobuQ at W1.58A2 surpasses BinaryDM and Bitnetv2 at W1.58A4, while providing a $17.3\times$ theoretical speedup and a $13.2\times$ compression ratio compared to the FP model. The visual

comparisons in Fig. 5 (right) further corroborate these results. More details are provided in Appendix C.

4.5. Zero-Shot Capability in W4A4 PTQ

Although primarily tailored for QAT, RobustQuantizer also works as a calibration-free W4A4 PTQ method. As shown in Table 4, it achieves the best FID with per-channel granularity, matching the accuracy of fine-grained SVDQuant while avoiding calibration data.

4.6. Additional Evaluation on Fast-Generation Models

To examine whether RobuQ remains effective beyond standard multi-step DiT sampling, we further evaluate it on representative fast-generation settings. As shown in Table 5, W4A4 PTQ changes FLUX-Schnell FID only from 18.40 to 18.57 under 4-step text-to-image generation. We also extend RobuQ to α -Flow, a one/two-step generative model, where W4A4 PTQ closely matches full precision at NFE=1 (2.59 vs. 2.58) and W1.58A4 QAT remains stable at NFE=1/2. These results indicate that low-bit quantization and few-step sampling are complementary: RobuQ reduces the cost of each model evaluation, while fast-generation models reduce the number of evaluations.

4.7. Scaling QAT and FID-50k Evaluation

We further scale QAT to 8 GPUs for W1.58A4/A3/A2 with batch size 256, learning rate 10^{-4} , and 100k training steps. As shown in Table 6, this setting reaches 4.55 FID at W1.58A4 without AMP and keeps A3/A2 below 10 FID. Notably, these ultra-low-bit DiT results require only about 1.5 days of training, which is encouraging for practical



Figure 6. RobuQ enables DiTs to generate competitive results at ultra-low bit settings. We show 256×256 samples from the W1.58A2 quantized model trained on ImageNet-1K.

Table 4. Performance comparison under W4A4 PTQ on ImageNet-1K 256×256 (steps=50, cfg=4.0).

Method	Granularity	Calib.	FID↓	Precision↑
FP (32/32)	-	-	19.11	0.9298
QueST	Channel	✓	84.20	0.2651
SVDQuant	Channel	✓	282.19	0.0098
SVDQuant	Group=64	✓	16.35	0.9087
RobustQuantizer	Channel	✗	16.13	0.9291

Table 5. Additional evaluation on fast-generation models. FLUX-Schnell is evaluated on MJHQ with 4 sampling steps, while α -Flow is evaluated on ImageNet-1K with one or two function evaluations (NFE).

Model / Method	W/A	FID↓	FID _{NFE=1} ↓	FID _{NFE=2} ↓
FLUX-Schnell FP	32/32	18.40	-	-
FLUX-Schnell + Ours	4/4	18.57	-	-
α -Flow-XL/2 FP	32/32	-	2.58	2.15
α -Flow-XL/2 + Ours	4/4	-	2.59	2.23
α -Flow-XL/2 + Ours	1.58/4	-	6.01	5.89

low-bit diffusion training.

5. Limitations and Future Work

RobuQ reduces theoretical computation and memory, but end-to-end acceleration still depends on low-bit kernels and hardware backends (Zhao et al., 2025b). In our current implementation, ternary weights are unpacked to an int4 backend, and the online Hadamard transform is fused only for non-absorbable projection layers. Thus, the measured

Table 6. FID-50k (cfg = 1.5) result of 8-GPU QAT on ImageNet-1K.

Setting	W/A	FID↓	sFID↓	IS↑
FP	32/32	3.71	7.85	242.21
RobuQ (w/o AMP)	1.58/4	4.55	7.26	194.96
RobuQ	1.58/3	5.04	7.85	187.27
RobuQ	1.58/2	8.47	10.24	145.90

speedup is still below the theoretical weighted-FLOPs reduction, especially for smaller DiT-XL/2 matrix shapes where packing and transformation overheads take a larger fraction of runtime. Our deployment stack also focuses on uniform W1.58A4 execution; dedicated mixed-precision A2/A3 kernels remain future work. We expect further gains from fused operators that jointly handle Hadamard transformation, per-token scaling, activation packing, low-bit GEMM, and the auxiliary low-rank branch.

6. Conclusion

We revisit quantization for Diffusion Transformers and identify activations as the key bottleneck for ultra-low-bit deployment. Building on a strong W1.58A4 baseline with an SVD-initialized low-rank branch and all-layer Hadamard mixing, we show that the Hadamard transform effectively regularizes per-token activation distributions and enables a distribution-agnostic RobustQuantizer. Combined with a hardware-friendly uniform implementation and activation-only mixed precision, RobuQ achieves stable training, improves generation quality, and establishes new state-of-the-art results for quantized DiTs, reaching W1.58A2.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (62501386, 625B1024), CCF-Tencent Rhino-Bird Open Research Fund, and CAAI-Tencent Rhino-Bird Open Research Fund. This work is also sponsored by AI Hundred Schools Program and is carried out using the Ascend AI technology stack.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. Quarot: Outlier-free 4-bit inference in rotated llms. In *NeurIPS*, 2024.
- Barratt, S. and Sharma, R. A note on the inception score. In *ICML Workshop*, 2018.
- Billingsley, P. *Probability and Measure*. 1995.
- Bobkov, S. G. Refinements of berry–esseen inequalities in terms of lyapunov coefficients. *Journal of Fourier Analysis and Applications*, 2023.
- Canonne, C. L. A short note on an inequality between kl and tv. *arXiv:2202.07198*, 2022.
- Chen, L., Meng, Y., Tang, C., Ma, X., Jiang, J., Wang, X., Wang, Z., and Zhu, W. Q-dit: Accurate post-training quantization for diffusion transformers. In *CVPR*, 2025.
- Chen, N., Zhang, Y., Zen, H., Weiss, R. J., Norouzi, M., and Chan, W. Wavegrad: Estimating gradients for waveform generation. In *ICLR*, 2020.
- Chen, Z., Zhang, Y., Liu, D., Xia, B., Gu, J., Kong, L., and Yuan, X. Hierarchical integration diffusion model for realistic image deblurring. In *NeurIPS*, 2023.
- Chen, Z., Qin, H., Guo, Y., Su, X., Yuan, X., Kong, L., and Zhang, Y. Binarized diffusion model for image super-resolution. In *NeurIPS*, 2024.
- Cover, T. M. and Thomas, J. A. *Elements of Information Theory*. 2006.
- Croitoru, F.-A., Hondru, V., Ionescu, R. T., and Shah, M. Diffusion models in vision: A survey. *TPAMI*, 2023.
- Csiszár, I. and Körner, J. *Information Theory: Coding Theorems for Discrete Memoryless Systems*. 2011.
- Dhariwal, P. and Nichol, A. Diffusion models beat gans on image synthesis. In *NeurIPS*, 2021.
- Feng, W., Qin, H., Yang, C., An, Z., Huang, L., Diao, B., Wang, F., Tao, R., Xu, Y., and Magno, M. Mpq-dm: Mixed precision quantization for extremely low bit diffusion models. In *AAAI*, 2025a.
- Feng, W., Yang, C., Qin, H., Li, Y., Li, X., An, Z., Huang, L., Diao, B., Zhuang, F., Magno, M., Xu, Y., Tian, Y., and Huang, T. Mpq-dmv2: Flexible residual mixed precision quantization for low-bit diffusion models with temporal distillation. In *arXiv preprint arXiv:2507.04290*, 2025b.
- Fino, B. and Algazi, V. Unified matrix treatment of the fast walsh–hadamard transform. *IEEE Transactions on Computers*, 1976.
- Gnedenko, B. V. and Kolmogorov, A. N. *Limit Distributions for Sums of Independent Random Variables*. 1954.
- He, C., Fang, C., Zhang, Y., Li, K., Tang, L., You, C., Xiao, F., Guo, Z., and Li, X. Reti-diff: Illumination degradation image restoration with retinex-based latent diffusion model. *arXiv preprint arXiv:2311.11638*, 2023.
- He, C., Shen, Y., Fang, C., Xiao, F., Tang, L., Zhang, Y., Zuo, W., Guo, Z., and Li, X. Diffusion models in low-level vision: A survey. *arXiv preprint arXiv:2406.11138*, 2024a.
- He, Y., Liu, J., Wu, W., Zhou, H., and Zhuang, B. Efficientdm: Efficient quantization-aware fine-tuning of low-bit diffusion models. In *ICLR*, 2024b.
- Heusel, M., Ransauer, H., Unterhiner, T., Nessler, B., and Hochreiter, S. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *NeurIPS*, 2017.
- Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv:2207.12598*, 2022.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Hu, S., Chen, L., Wu, P., Li, H., Yan, J., and Tao, D. St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *ECCV*, 2022.
- Hu, X., Cheng, Y., Yang, D., Xu, Z., Yuan, Z., Yu, J., Xu, C., Jiang, Z., and Zhou, S. Ostquant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting. In *ICLR*, 2025.
- Hwang, Y., Lee, H., and Kang, J. Tq-dit: Efficient time-aware quantization for diffusion transformers. In *arXiv preprint arXiv:2502.04056*, 2025.

- Jaynes, E. T. Information theory and statistical mechanics. 1957.
- K Esser, S., L McKinstry, J., Bablani, D., Appuswamy, R., and S Modha, D. Learned step size quantization. In *ICLR*, 2019.
- Karras, T., Laine, S., and Aila, T. A style-based generator architecture for generative adversarial networks. In *CVPR*, 2019.
- Kim, D., Hwang, J., Oh, C., and Park, J. Mixdit: Accelerating image diffusion transformer inference with mixed-precision mx quantization. In *arXiv preprint arXiv:2504.08398*, 2025.
- Kolb, C., Müller, C. L., Bisch, B., and Rügamer, D. Smoothing the edges: Smooth optimization for sparse regularization using hadamard overparametrization. In *arXiv preprint arXiv:2307.03571*, 2023.
- Labs, B. F. Flux.1. 2024. URL <https://blackforestlabs.ai>.
- Li, M., Lin, Y., Zhang, Z., Cai, T., Li, X., Guo, J., Xie, E., Meng, C., Zhu, J.-Y., and Han, S. Svdquant: Absorbing outliers by low-rank components for 4-bit diffusion models. In *ICLR*, 2025.
- Li, Y., Xiong, H., Kong, L., Sun, Z., Chen, H., Wang, S., and Yin, D. Mpgraf: a modular and pre-trained graphformer for learning to rank at web-scale. In *ICDM*, 2023a.
- Li, Y., Xiong, H., Kong, L., Zhang, R., Xu, F., Chen, G., and Li, M. Mhrr: Moocs recommender service with meta hierarchical reinforced ranking. *TSC*, 2023b.
- Li, Y., Wang, H., Jin, Q., Hu, J., Chemerys, P., Fu, Y., Wang, Y., Tulyakov, S., and Ren, J. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. In *NeurIPS*, 2024.
- Lin, H., Xu, H., Wu, Y., Cui, J., Zhang, Y., Mou, L., Song, L., Sun, Z., and Wei, Y. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. In *NeurIPS*, 2025.
- Liu, C., Wu, H., Zhong, Y., Zhang, X., Wang, Y., and Xie, W. Intelligent grimm-open-ended visual storytelling via latent diffusion models. In *CVPR*, 2024.
- Liu, K., Yang, K., Chen, Z., Li, Z., Guo, Y., Li, W., Kong, L., and Zhang, Y. Bimacosr: Binary one-step diffusion model leveraging flexible matrix compression for real super-resolution. In *ICML*, 2025a.
- Liu, W. and Zhang, S. Q. Hq-dit: Efficient diffusion transformer with fp4 hybrid quantization. In *arXiv preprint arXiv:2405.19751*, 2024.
- Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. Spinquant: Llm quantization with learned rotations. In *ICLR*, 2025b.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *ICLR*, 2019.
- Lu, X., Zhou, A., Lin, Z., Liu, Q. and Xu, Y., Zhang, R., Yang, X., Yan, J., Gao, P., and Li, H. Terdit: Ternary diffusion models with transformers. In *arXiv preprint arXiv:2405.14854*, 2024.
- Lv, C., Chen, H., Guo, J., Ding, Y., and Liu, X. Ptq4sam: Post-training quantization for segment anything. In *CVPR*, 2024.
- Lyons, R. and Zumbun, K. A calculus proof of the cramér-wold theorem. *Proceedings of the American Mathematical Society*, 2017. arXiv:1607.03206.
- Ma, S., Wang, H., Ma, L., Wang, Lei and Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., and Wei, F. The era of 1-bit llms: All large language models are in 1.58 bits. In *arXiv preprint arXiv:2402.17764*, 2024.
- Nash, C., Menick, J., Dieleman, S., and Battaglia, P. W. Generating images with sparse representations. In *arXiv preprint arXiv:2103.03841*, 2021.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *ICCV*, 2023.
- Petersen, K. B. and Pedersen, M. S. The matrix cookbook, 2012.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al. Imagenet large scale visual recognition challenge. *IJCV*, 2015.
- Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X. Improved techniques for training gans. In *NeurIPS*, 2016.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *NeurIPS*, 2017.
- Wang, H., Ma, S., and Wei, F. Bitnet v2: Native 4-bit activations with hadamard transformation for 1-bit llms. In *arXiv preprint arXiv:2504.18415*, 2025a.
- Wang, H., Shang, Y., Yuan, Z., Wu, J., and Yan, Y. Quest: Low-bit diffusion model quantization via efficient selective finetuning. In *ICCV*, 2025b.
- Wu, J., Wang, H., Shang, Y., Shah, M., and Yan, Y. Ptq4dit: Post-training quantization for diffusion transformers. In *NeurIPS*, 2024.
- Xu, Z., Zhao, Z., Hu, X., Chen, Z., and Yang, D. KBVQ-MoE: KLT-guided SVD with bias-corrected vector quantization for MoE large language models. In *ICLR*, 2026.
- Yang, L., Zhang, Z., Song, Y., Hong, S., Xu, R., Zhao, Y., Zhang, W., Cui, B., and Yang, M.-H. Diffusion models: A comprehensive survey of methods and applications. In *ACM Computing Surveys*, 2023.
- Yarlagadda, R. and Hershey, J. *Hadamard Matrix Analysis and Synthesis: With Applications to Communications and Signal/Image Processing*. 1993.
- Zhang, Y., Qin, H., Zhao, Z., Liu, X., Danelljan, M., and Yu, F. Flexible residual binarization for image super-resolution. In *ICML*, 2024.
- Zhao, T., Ning, X., Fang, T., Liu, E., Huang, G., Lin, Z., Shengen, Y., Dai, G., and Wang, Y. Mixdq: Memory-efficient few-step text-to-image diffusion models with metric-decoupled mixed precision quantization. In *ECCV*, 2024a.
- Zhao, T., Fang, T., Huang, H., Liu, E., Wan, R., Soedarmadji, W., Li, S., Lin, Z., Dai, G., Yan, S., Yang, H., Nong, X., and Wang, Y. Vedit-q: Efficient and accurate quantization of diffusion transformers for image and video generation. In *ICLR*, 2025a.
- Zhao, W., Wang, H., Zhou, J., and Lu, J. Dc-solver: Improving predictor-corrector diffusion sampler via dynamic compensation. In *arXiv preprint arXiv:2409.03755*, 2024, 2024b.
- Zhao, Z., Li, H., Liu, F., Lu, Y., Wang, Z., Yang, T., Jiang, L., and Guan, H. QUARK: Quantization-enabled circuit sharing for transformer acceleration by exploiting common patterns in nonlinear operations. In *ICCAD*, 2025b.
- Zhao, Z., Liu, F., Wang, J., Guan, C., Wang, Z., Jiang, L., and Guan, H. SpecQuant: Spectral decomposition and adaptive truncation for ultra-low-bit LLMs quantization. In *AAAI*, 2026a.
- Zhao, Z., Xu, Z., and Yang, D. BWLA: Breaking the barrier of W1AX post-training quantization for LLMs. In *ACL*, 2026b.
- Zheng, X., Liu, X., Bian, Y., Ma, X., Zhang, Y., Wang, J., Guo, J., and Qin, H. Bidm: Pushing the limit of quantization for diffusion models. In *NeurIPS*, 2024.
- Zheng, X., Liu, X., Qin, H., Ma, X., Zhang, M., Hao, H., Wang, J., Zhao, Z., Guo, J., and Magno, M. Binarydm: Accurate weight binarization for efficient diffusion models. In *ICLR*, 2025.
- Zhou, S., Wu, Y., Ni, Z., Zhou, X., Wen, H., and Zou, Y. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *ICLR*, 2016.

Contents of Appendix

A Formal Proof: Hadamard Transform Produces Approximately Normal Coordinates

- A.1 Exact Second-Moment Identities
- A.2 Central Limit Theorem and Asymptotic Independence
- A.3 Off-Diagonal Covariance Decay
- A.4 Quantitative Closeness to a Product Gaussian
- A.5 Quantization and Mean-Squared Error

B Activation Mixed Precision Network Pipeline

C Efficiency Analysis and Deployment

- C.1 FLOPs and Memory Breakdown in DiT-XL/2 Model
- C.2 Calculate FLOPs of RobuQ W1.58A4 Model
- C.3 Integration of the Hadamard Transform
- C.4 Deployment

D Additional Analysis

- D.1 Ablation Study on SVD Rank
 - D.2 Comparison of Uniform and Non-Uniform Quantizers
 - D.3 Mixed-Precision Analysis
 - D.4 Visualization Results Comparisons
-

A. Formal Proof: Hadamard Transform Produces Approximately Normal Per-Token Coordinates

Notation aligned with the main text. Let a single token's activation vector be denoted by $\mathbf{x} \in \mathbb{R}^C$ (token index t suppressed for clarity). We use the normalized Hadamard transform (Yarlagadda & Hershey, 1993)

$$H \in \{\pm \frac{1}{\sqrt{C}}\}^{C \times C}, \quad H^\top H = HH^\top = I_C, \quad (14)$$

and define the transformed coordinates $\mathbf{y} = H\mathbf{x}$. For channel j , set

$$\mu_j = \mathbb{E}[x_j], \quad \tilde{x}_j := x_j - \mu_j, \quad \sigma_j^2 = \text{Var}(x_j), \quad (15)$$

and define the per-token average variance

$$\sigma_t^2 := \frac{1}{C} \sum_{j=1}^C \sigma_j^2. \quad (16)$$

We also write the Hadamard coefficients as $a_j^{(c)} := H_{cj} = \pm \frac{1}{\sqrt{C}}$, so that

$$y_c = \sum_{j=1}^C a_j^{(c)} x_j, \quad c = 1, \dots, C. \quad (17)$$

A.1. Exact Second-Moment Identities

By linearity and orthogonality (distribution-free), for each coordinate c ,

$$\mathbb{E}[y_c] = \sum_{j=1}^C a_j^{(c)} \mu_j, \quad (18)$$

$$\text{Var}(y_c) = \sum_{j=1}^C (a_j^{(c)})^2 \sigma_j^2 = \frac{1}{C} \sum_{j=1}^C \sigma_j^2 = \sigma_t^2, \quad (19)$$

$$\text{Cov}(y_c, y_{c'}) = \sum_{j=1}^C a_j^{(c)} a_j^{(c')} \sigma_j^2 = \frac{1}{C} \sum_{j=1}^C s_j^{(c,c')} \sigma_j^2, \quad s_j^{(c,c')} := \text{sign}(H_{cj} H_{c'j}) \in \{\pm 1\}. \quad (20)$$

Equation 19 shows exact variance equalization across transformed channels; (20) expresses off-diagonals as signed averages of per-channel variances.

A.2. Central Limit Theorem and Asymptotic Independence

Assumptions (A1–A3).

- (A1) The centered variables $\tilde{x}_j$ are independent (or weakly dependent in a manner admitting triangular-array CLTs (Gnedenko & Kolmogorov, 1954)).
- (A2) There exists $\kappa > 0$ with $\sup_j \mathbb{E}[|\tilde{x}_j|^{2+\kappa}] < \infty$; in particular $\sup_j \mathbb{E}[|\tilde{x}_j|^3] \leq M_3 < \infty$.
- (A3) No adversarial alignment of $\{\mu_j\}, \{\sigma_j^2\}$ with Hadamard sign patterns (practically, variance deviations are not aligned with a single Hadamard row/column).

Univariate CLT (Berry–Esseen). Fix c and consider the triangular-array terms $\xi_j^{(C)} := a_j^{(c)} \tilde{x}_j$. Their variance sum is

$$s_C^2 = \sum_{j=1}^C \text{Var}(\xi_j^{(C)}) = \sum_{j=1}^C (a_j^{(c)})^2 \sigma_j^2 = \sigma_t^2. \quad (21)$$

Because $|a_j^{(c)}| = 1/\sqrt{C}$,

$$\sum_{j=1}^C \mathbb{E}[|\xi_j^{(C)}|^3] = \frac{1}{C^{3/2}} \sum_{j=1}^C \mathbb{E}[|\tilde{x}_j|^3] \leq \frac{M_3}{\sqrt{C}}. \quad (22)$$

Berry–Esseen for non-identical summands yields an absolute constant K_{BE} such that

$$\sup_{x \in \mathbb{R}} \left| \Pr\left(\frac{\sum_{j=1}^C \xi_j^{(C)}}{\sigma_t} \leq x\right) - \Phi(x) \right| \leq \frac{K_{\text{BE}} M_3}{\sigma_t^3 \sqrt{C}}. \quad (23)$$

Thus each scalar coordinate (after centering and normalization) converges to $\mathcal{N}(0, 1)$ with Kolmogorov error $O(C^{-1/2})$ (Bobkov, 2023).

Finite-Dimensional Gaussian Convergence. For fixed indices $c_1, \dots, c_m$ (with m independent of C) and any $\lambda \in \mathbb{R}^m$,

$$L_C(\lambda) := \sum_{r=1}^m \lambda_r \frac{\sum_{j=1}^C a_j^{(c_r)} \tilde{x}_j}{\sigma_t} = \frac{1}{\sigma_t} \sum_{j=1}^C \left(\sum_{r=1}^m \lambda_r a_j^{(c_r)} \right) \tilde{x}_j, \quad (24)$$

where the inner coefficients are $O(C^{-1/2})$ uniformly in j . Standard Lyapunov/Lindeberg conditions hold, implying

$$L_C(\lambda) \xrightarrow{d} \mathcal{N}(0, \lambda^\top \Lambda \lambda), \quad (25)$$

with limit covariance Λ determined by (20). By Cramér–Wold, $(y_{c_1}, \dots, y_{c_m})$ converges to a multivariate Gaussian whose diagonal entries equal σ_t^2 (Billingsley, 1995; Lyons & Zumbun, 2017).

A.3. Off-Diagonal Covariance Decay and Asymptotic Independence

Write variance deviations $\delta_j := \sigma_j^2 - \sigma_t^2$. From (20),

$$\text{Cov}(y_c, y_{c'}) = \frac{1}{C} \sum_{j=1}^C s_j^{(c, c')} \delta_j. \quad (26)$$

Orthogonality of Hadamard rows implies near-cancellation of the ± 1 signs in the average; two practical sufficient conditions ensuring $\text{Cov}(y_c, y_{c'}) \rightarrow 0$ as $C \rightarrow \infty$ are:

- *Uniform small deviations:* $\max_j |\delta_j| \rightarrow 0 \Rightarrow |\text{Cov}(y_c, y_{c'})| \leq \max_j |\delta_j| \rightarrow 0$.
- ℓ_2 -small deviations: letting $\mathbf{w}^{(c, c')} = (s_1^{(c, c')}, \dots, s_C^{(c, c')})$,

$$|\text{Cov}(y_c, y_{c'})| = \frac{1}{C} |\langle \boldsymbol{\delta}, \mathbf{w}^{(c, c')} \rangle| \leq \frac{\|\boldsymbol{\delta}\|_2}{\sqrt{C}}, \quad \Rightarrow \|\boldsymbol{\delta}\|_2 = o(\sqrt{C}) \implies \text{Cov}(y_c, y_{c'}) \rightarrow 0. \quad (27)$$

Combined with the finite-dimensional CLT, this yields asymptotic joint Gaussianity with diagonal covariance $\sigma_t^2 I_m$; hence the transformed coordinates become asymptotically independent Gaussians.

A.4. Quantitative Closeness to a Product Gaussian: KL and TV Bounds

Let Σ_m be the covariance of $(y_{c_1}, \dots, y_{c_m})$ and decompose

$$\Sigma_m = \sigma_t^2 I_m + E_m, \quad (28)$$

where E_m has zeros on the diagonal and off-diagonals $e_{ij} = \text{Cov}(y_{c_i}, y_{c_j})$. Then

$$\text{KL}(\mathcal{N}(\mu_m, \Sigma_m) \parallel \mathcal{N}(\mu_m, \sigma_t^2 I_m)) = -\frac{1}{2} \ln \det(I_m + \sigma_t^{-2} E_m). \quad (29)$$

If $\|\sigma_t^{-2} E_m\|_{\text{op}} < \frac{1}{2}$ (Petersen & Pedersen, 2012), expand $\ln \det(I + A)$ to obtain

$$\text{KL} = \frac{1}{4} \sigma_t^{-4} \|E_m\|_F^2 + O(\|E_m\|_F^3 / \sigma_t^6). \quad (30)$$

Using $\|E_m\|_F^2 \leq m(m-1) \max_{i \neq j} e_{ij}^2$ and $|e_{ij}| \leq \|\delta\|_2 / \sqrt{C}$ from Section A.3,

$$\text{KL} = O\left(\frac{m^2 \|\delta\|_2^2}{C \sigma_t^4}\right). \quad (31)$$

By Pinsker (Csiszár & Körner, 2011; Canonne, 2022), $\text{TV} \leq \sqrt{\frac{1}{2} \text{KL}}$, hence

$$\text{TV}(\mathcal{N}(\mu_m, \Sigma_m), \mathcal{N}(\mu_m, \sigma_t^2 I_m)) = O\left(\frac{m \|\delta\|_2}{\sqrt{C} \sigma_t^2}\right). \quad (32)$$

The total deviation of the true law of $(y_{c_1}, \dots, y_{c_m})$ from a product Gaussian equals the multivariate non-Gaussianity error (Berry–Esseen/Bentkus type, $O(C^{-1/2})$) plus (32). Thus, for fixed m ,

$$\text{TV}_{\text{total}} = O(C^{-1/2}) + O\left(\frac{m \|\delta\|_2}{\sqrt{C} \sigma_t^2}\right), \quad (33)$$

which vanishes at rate $O(C^{-1/2})$ when $\|\delta\|_2 = o(\sqrt{C})$.

A.5. Quantization and Mean-Squared Error

Under the mean-squared error (MSE) metric, applying the Hadamard transform for quantization does not change the final quantization error. This conclusion follows from the orthogonality of the Hadamard matrix (after normalization).

Let the activation vector be X , and the transformed vector be $Y = HX$. If we quantize Y to get $Q(Y)$ and then recover the vector via the inverse transform, the resulting vector is $X_{\text{rec}} = H^\top Q(Y) = H^\top Q(HX)$.

The MSE of the quantization error is:

$$\text{MSE} = \mathbb{E}[\|X - X_{\text{rec}}\|_2^2] = \mathbb{E}[\|X - H^\top Q(HX)\|_2^2] \quad (34)$$

Since an orthogonal transform preserves the Euclidean norm (length) of a vector, we have:

$$\text{MSE} = \mathbb{E}[\|H(X - H^\top Q(HX))\|_2^2] = \mathbb{E}[\|HX - HH^\top Q(HX)\|_2^2] = \mathbb{E}[\|HX - Q(HX)\|_2^2] \quad (35)$$

This shows that $\text{MSE} = \mathbb{E}[\|Y - Q(Y)\|_2^2]$. This identity demonstrates that the mean-squared error of quantizing the original vector X is identical to the mean-squared error of quantizing the transformed vector Y . This means our objective can shift from “how to quantize X ” to “how to quantize Y .”

As proven in this document, the coordinates of Y are approximately Gaussian and nearly independent. This provides a great convenience for designing a quantizer. We can now transform a complex multivariate quantization problem into quantizing a series of approximately independent Gaussian variables.

The Gaussian distribution has the highest entropy among all continuous distributions with a given variance (Cover & Thomas, 2006; Jaynes, 1957). From an information-theoretic perspective, this means it contains the maximum randomness or “uncertainty.” Therefore, for a given number of quantization bits, quantizing a Gaussian distribution is the “most difficult” task and typically results in the largest quantization error. Our method effectively prepares for this “worst-case” scenario.

By designing a quantizer optimized for the Gaussian distribution, we ensure that the quantization scheme is robust and effective for the Hadamard-transformed activations under the MSE metric.

Fundamental Insight: Distribution-Agnostic Quantization via Whitening

Core Idea: We note that a lot existing quantization works focus on observing data distributions to extract prior knowledge and design corresponding quantizers, but what if we *erase* prior knowledge instead?

The RobustQuantizer Paradigm:

- No Prior Assumptions: Instead of modeling input statistics, we use random orthogonal projections to *actively transform* inputs into the worst-case distribution – Gaussian noise
- Embracing the Hardest Case: While $\mathcal{N}(0, 1)$ has minimal information (maximum entropy), its perfect knownness allows pre-computing optimal quantization parameters
- Theoretical Guarantee: This establishes a rigorous *lower-bound* for quantization performance *without requiring any prior knowledge* about input distributions

Future Work:

Exploring alternative transformation methods to *other known distributions* and investigating their trade-offs between information preservation and quantization efficiency will be our key focus for future work.

B. Activation Mixed Precision Network Pipeline

We summarize the AMPN pipeline succinctly and provide compact pseudocode for the major algorithmic components. Here, L is the number of activation layers, $b_\ell \in \mathcal{B} = \{1, 2, 3, 4\}$ is the chosen activation bit-width for layer ℓ , weights are frozen to ternary (W1.58), and w_ℓ denotes the FLOPs-based cost weight for layer ℓ with $W_{\text{tot}} = \sum_\ell w_\ell$. For a single-layer QAT run let $\Delta L_\ell(b_l) = L_{\ell, b_l} - L_{\text{FP}}$ be the validation loss gap after short training; the constrained objective is

$$\min_{\{b_\ell\}} \sum_{\ell=1}^L \Delta L_\ell(b_\ell) \quad \text{s.t.} \quad \frac{1}{W_{\text{tot}}} \sum_{\ell=1}^L w_\ell b_\ell \leq \bar{B}_{\text{tgt}}. \quad (36)$$

The pipeline is as follows:

1. Run Algorithm 1 to obtain the QAT-based sensitivity table $\Delta L_\ell(b_l)$. This involves briefly training each layer individually at a given bit-width while all other layers are frozen in FP.
2. Generate the optimal bit-width allocation C_{dp} by running Algorithm 2 with the sensitivity table from the previous step.
3. Train the selected allocation C_{dp} end-to-end with a full QAT schedule.

Algorithm 1 QAT-based Sensitivity Profiling

Require: FP model $\mathcal{M}$, validation pool $\mathcal{V}_{\text{pool}}$, training data $\mathcal{D}_{\text{train}}$, bit set $\mathcal{B}$, short QAT steps T_{short}

Ensure: sensitivity table $\Delta L_\ell(b_l)$ for all ℓ, b_l

```

1:  $L_{\text{FP}} \leftarrow \text{Eval}(\mathcal{M}, \mathcal{V}_{\text{pool}})$ 
2: for each layer  $\ell = 1, \dots, L$  do
3:   for each  $b_l \in \mathcal{B}$  do
4:      $\mathcal{M}_q \leftarrow \text{Copy}(\mathcal{M})$ 
5:     Quantize layer  $\ell$  of  $\mathcal{M}_q$  to  $b_l$  bits.
6:     Freeze parameters of all layers  $\ell' \neq \ell$  in  $\mathcal{M}_q$ .
7:     Train  $\mathcal{M}_q$  for  $T_{\text{short}}$  steps on  $\mathcal{D}_{\text{train}}$ .
8:      $L_{\ell, b_l} \leftarrow \text{Eval}(\mathcal{M}_q, \mathcal{V}_{\text{pool}})$ .
9:      $\Delta L_\ell(b_l) \leftarrow L_{\ell, b_l} - L_{\text{FP}}$ .
10:  end for
11: end for  $\Delta L_\ell(b_l)$ 
```


Algorithm 2 Discretized DP for Bit-Width Allocation**Require:** sensitivity table $\Delta L_\ell(b_l)$ for $\ell \in \mathcal{L}_{dp}$, layer costs w_ℓ , resolution β , target $\bar{B}_{tgt}$ **Ensure:** DP-optimal allocation C_{dp} on $\mathcal{L}_{dp}$

```

1: Compute total DP cost  $W_{tot}^{dp} \leftarrow \sum_{\ell \in \mathcal{L}_{dp}} w_\ell$ 
2: for each  $\ell \in \mathcal{L}_{dp}$  do
3:   Discretize layer cost:  $\Delta w_\ell \leftarrow \left\lfloor \beta \frac{w_\ell}{W_{tot}^{dp}} \right\rfloor$ 
4: end for
5: Discretize target budget:  $B \leftarrow \left\lfloor \beta \bar{B}_{tgt} \right\rfloor$ 
6: Initialize  $DP[0 \dots |\mathcal{L}_{dp}|][0 \dots B] \leftarrow +\infty$ ;  $DP[0][0] \leftarrow 0$ 
7: for  $i = 1$  to  $|\mathcal{L}_{dp}|$  do
8:   Let  $\ell$  be the  $i$ -th layer in  $\mathcal{L}_{dp}$ 
9:   for  $w = 0$  to  $B$  do
10:    for each  $b_l \in \mathcal{B}$  do
11:       $w' \leftarrow w + \Delta w_\ell \cdot b_l$ 
12:      if  $w' \leq B$  then
13:         $DP[i][w'] \leftarrow \min(DP[i][w'], DP[i-1][w] + \Delta L_\ell(b_l))$ 
14:      end if
15:    end for
16:  end for
17: end for
18: Backtrack from  $\arg \min_{w \leq B} DP[|\mathcal{L}_{dp}|][w]$  to recover allocation  $C_{dp}$ .  $C_{dp}$ 

```

	Embedding	DiT Blocks					Final Layer
$x^{4 \times 32 \times 32}$	X embedding	Attention		MLP		AdaLN - Zero	Final layer
t	Label embedding						Unpatchify
y	Timestep embedding						
	Position embedding	qkv	QK	fc1	fc2		
		proj	SV				
GFLOPs	0.0016	36.71	4.053	36.71	36.71	0.2026	0.1268
MB	11.79	567.4	0	567.4	567.4	851.1	10.27

Figure 7. FLOPs and Memory Breakdown in DiT-XL/2 Model.

C. Efficiency Analysis and Deployment**C.1. FLOPs and Memory Breakdown in DiT-XL/2 Model**

Here we provide an analysis of the FLOPs and memory usage for the DiT-XL/2 model (Peebles & Xie, 2023), as shown in Figure 7. As illustrated, the DiT block accounts for the vast majority of FLOPs and memory consumption ($\geq 99\%$). Therefore, we keep the embedding section and the final layer at FP without quantization. Within the DiT block, the MLP and adaLN-zero modules occupy most of the memory ($\geq 77\%$), while the MLP and attention components dominate the FLOPs ($\geq 99\%$). When categorized by computation type, the primary computations occur between weights and activations ($\geq 96\%$). In contrast, operations between activations and activations constitute a small proportion but have a significant impact, so we maintain these operations at **8-bit** precision.

C.2. Calculate FLOPs of RobuQ W1.58A4 Model

We employ FLOPs as metrics for evaluating theoretical inference efficiency. For quantization operations, we define the weighted FLOPs as follows:

$$\text{FLOPs}(W = 1.58, A = N) = \frac{1}{2} \cdot \text{FLOPs}(W = N, A = N) = \frac{N}{32} \cdot \text{FLOPs} \quad (37)$$

For the Hadamard transform, since it possesses a fast algorithm with $\mathcal{O}(n^2 \log n)$ complexity (Fino & Algazi, 1976) and can be absorbed into the weight matrix within a DiT block—ultimately requiring only four online Hadamard transforms—its theoretical computational cost is negligible. We emphasize that the weighted FLOPs below measure theoretical arithmetic reduction rather than end-to-end wall-clock speed; the latter also depends on kernel fusion, packing overhead, and the available low-bit backend. We provide FLOPs breakdown in RobuQ (w/o AMP) W1.58A4 model as Table 7 shown.

Table 7. Flops breakdown in RobuQ (w/o AMP) W1.58A4 DiT-XL/2 model.

	Embedding	Low rank branch	A-A Matrix Multiplication	W-A Matrix Multiplication	Final Layer	Total
bits/bits	32/32	32/32	8/8	1.58/4	32/32	N/A
GFLOPs (G)	0.0016	2.0312	1.0133	6.9213	0.1268	10.07

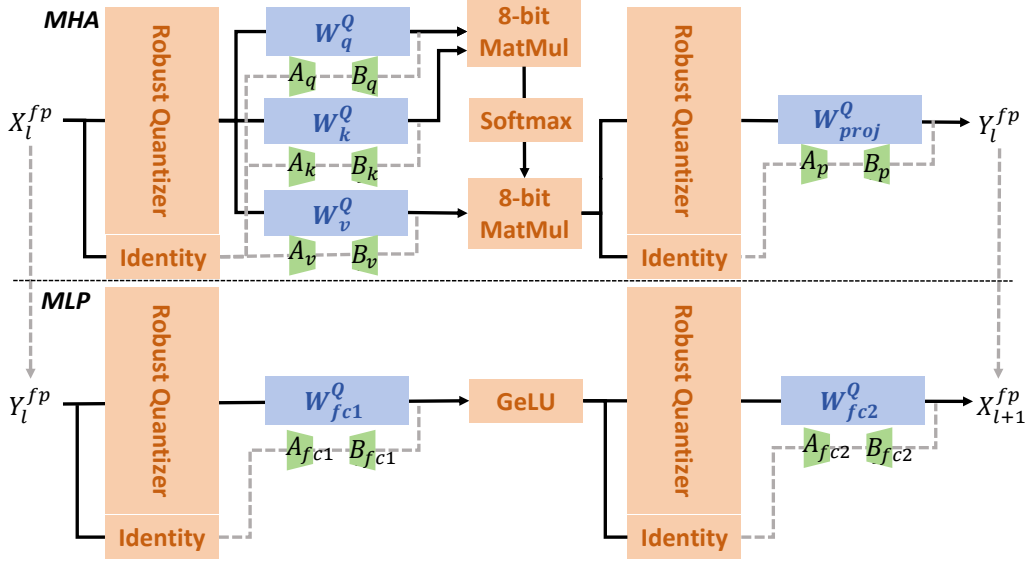


Figure 8. Schematic diagram of actual deployment. For simplicity, we have omitted the AdaLN-zero components to highlight the sections accounting for the majority of FLOPs.

C.3. Integration of the Hadamard Transform During Inference

For completeness, we describe how the Hadamard transform is incorporated into the inference pipeline. In our implementation, the transform is applied online only to the activations of *mlp.fc2* and *attn.proj*, while all other occurrences can be absorbed into the weights of preceding layers, ensuring no additional runtime overhead elsewhere. We implement the operation using the **fast-hadamard-transformer** library, where the Hadamard computation is largely memory-bound and typically accounts for only 5%–10% of the total time of the associated matrix operations.

Although additional speedups are achievable—for example, by leveraging the butterfly structure of the Hadamard transform to fuse it with subsequent low-bit operators—such improvements require specialized kernel engineering and are left for future optimization. We report end-to-end deployment measurements in Sec. C.4, while leaving fully fused kernel-level optimization to future work.

C.4. Deployment

Our proposed RobuQ scheme is primarily motivated by deployment efficiency, but we separate weighted FLOPs from measured wall-clock speed. In the current implementation, ternary weights are packed by compacting five values into an Int8 and are unpacked to an int4 backend during inference using SVDQuant’s (Li et al., 2025) Nunchaku framework. Therefore, the W1.58A4 throughput should be interpreted as an implementation on top of an int4 execution backend rather than a fully specialized ternary kernel. As shown in Table 8, RobuQ reduces the checkpoint size from 2.58GB to 0.17GB (15.2 $\times$), lowers peak memory from 3,914MB to 554MB (7.1 $\times$), and improves end-to-end iteration speed from 44.09 to 155.37 iter/s (3.52 $\times$). Removing the online Hadamard transform increases throughput to 168.42 iter/s, indicating that Hadamard contributes only about 7.7% overhead in this deployment.

The remaining gap between weighted-FLOPs reduction and wall-clock speed mainly comes from model-level and kernel-level factors. DiT-XL/2 has relatively small GEMM shapes, where front-end packing, activation scaling, and online transformation occupy a larger fraction of runtime. As matrix sizes grow, GEMM becomes dominant and the measured speedup approaches the theoretical low-bit regime, as shown in Table 9. We also measure FLUX-Schnell v2, whose hidden dimension and sequence length are larger than DiT-XL/2; adding online Hadamard changes per-step latency only from 0.2166s to 0.2202s, with nearly unchanged peak memory (Table 10). These results suggest that the DiT-XL/2 deployment number is a conservative lower bound for larger backbones.

Table 8. Deployment on NVIDIA RTX 4090. Iteration Speed is calculated with batchsize=8.

Method	W/A	FLOPs	Checkpoint Size	Max Memory Allocated	Iteration Speed
FP	32/32	114.52	2.58GB	3,914MB	44.09 iter/s
RobuQ	1.58/4	10.07	0.17GB	554MB	155.37 iter/s
RobuQ (w/o Hadamard)	1.58/4	10.07	0.17GB	551MB	168.42 iter/s

Table 9. Deployment scaling analysis. Larger matrices reduce the relative front-end overhead and approach the low-bit GEMM regime.

Setting	E2E	Linear	None	Had.	Fused	Front-end / GEMM / Total (ms)
DiT shape (16, 256, 4608, 1152)	-	-	5.14×	3.37×	3.83×	0.210 / 0.084 / 0.305
Large shape (16, 1152, 8192, 8192)	-	-	12.03×	9.29×	11.97×	0.615 / 4.226 / 4.828
Wide DiT hidden=2560	4.45×	6.39×	-	-	-	-
Wide DiT hidden=4096	6.12×	8.74×	-	-	-	-

For W1.58A2/A3, our current work demonstrates stable generation and strong quality, but we do not claim a fully optimized mixed-precision deployment stack. Developing dedicated mixed-bit activation kernels is left as future work; once such kernels are available, the bit-width allocation introduces no additional training or inference-time profiling overhead.

D. Additional Analysis

D.1. Ablation Study on SVD Rank

To further understand the impact of the low-rank decomposition used in our weight processing pipeline, we conduct an ablation study by varying the SVD rank under the W1.58A4 configuration. All models are trained for 200k steps using 10k samples (cfg = 1.5, step size = 50), and evaluated under the same protocol as our main experiments. The results are reported in Table 11.

We observe that increasing the rank consistently improves generative quality, especially in IS and FID. However, the marginal gain becomes smaller when moving from rank 16 to rank 32, while the computational overhead continues to increase. Based on this trade-off between accuracy and efficiency, we adopt rank = 16 in all main experiments.

D.2. Comparison of Uniform and Non-Uniform Quantizers on T2I Models

To further examine whether the advantages of our Gaussian-optimized non-uniform quantizer persist when applied to larger and more advanced text-to-image (T2I) models, we conduct an additional experiment on a recent state-of-the-art open-source model, FLUX (Labs, 2024). Specifically, we evaluate the Flux-Schnell variant using 4 sampling steps on 5k images from the MJHQ dataset. All settings remain consistent across quantizers to ensure a fair comparison. The quantitative results are reported in Table 12.

Across all metrics, the non-uniform quantizer demonstrates a mild but consistent improvement over the uniform quantizer, aligning with our theoretical motivation based on Gaussianity after the Hadamard transform. These gains, however, remain relatively modest in magnitude.

In practice, the choice of quantizer must also consider factors beyond accuracy alone. Compared to its non-uniform counterpart, the uniform quantizer exhibits stronger robustness across heterogeneous hardware backends, simpler implementation, and lower computational overhead—particularly for high-throughput inference workloads. Taking these engineering considerations into account, we ultimately select the **Uniform Quantizer** as the preferred option for real-world deployment, despite the slight empirical advantage of non-uniform quantization on larger models such as FLUX.

D.3. Mixed-Precision Analysis

Setup. With *adaLN* fixed to 4-bit, we examine activation bit allocation only on learnable layers. Fig. 9 (left) shows per-block heatmaps for the four ops (*attn.qkv*, *attn.proj*, *mlp.fc1*, *mlp.fc2*) under two activation budgets, W1.58A2 and W1.58A3. Fig. 9 (right) summarizes the mean per-op allocation via pie charts, while Fig. 10 (top) and Fig. 10 (bottom) plot, respectively, the per-block average activation bits and the normalized per-block activation share.

Findings.

- **Attention consumes the budget first.** From the heatmaps (Fig. 9 (left)), attention paths (*attn.qkv*, *attn.proj*) retain higher bitwidths in mid and late blocks under W1.58A2. When moving from A2 to A3, the bit-width allocation becomes more balanced across all ops, with extra capacity used to maintain higher precision, although attention still dominates.

Table 10. Online Hadamard overhead on FLUX-Schnell v2.

Setting	Per-step latency	Peak memory	Overhead
w/o Hadamard	0.2166s	15.95GB	-
w/ Hadamard	0.2202s	15.98GB	$\sim 1.7\%$

Table 11. Ablation study on different SVD ranks for W1.58A4 quantization.

Rank	IS $\uparrow$	FID $\downarrow$	sFID $\downarrow$	Precision $\uparrow$
8	87.43	23.21	29.96	0.6327
16	91.28	22.25	29.24	0.6328
32	92.11	21.71	28.89	0.6324

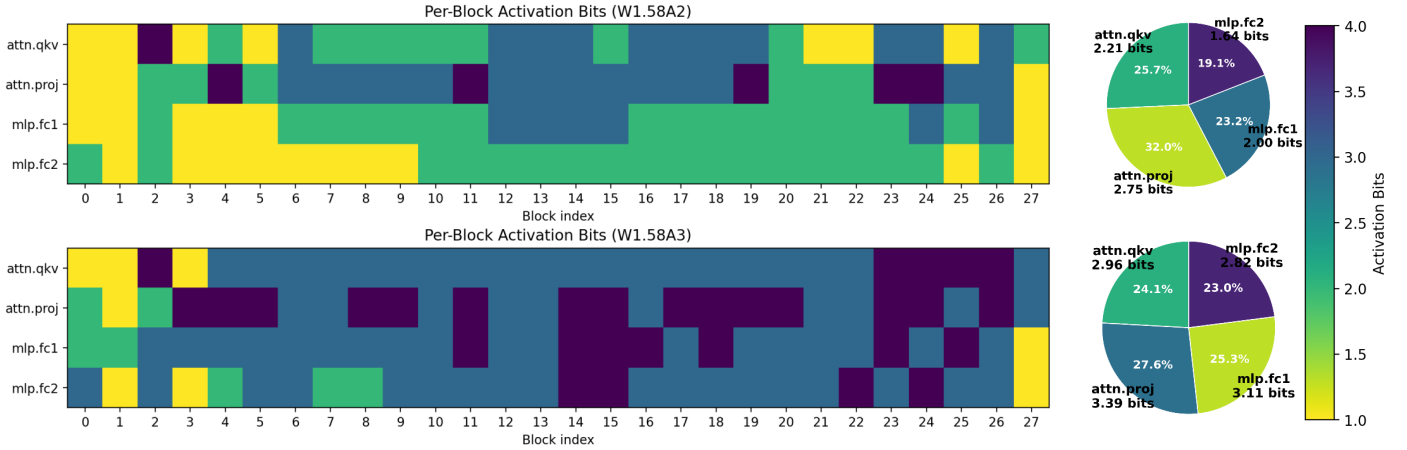


Figure 9. Visualization of Activation Bit-Width Distribution

- ***attn.proj* is the largest sink under tight budgets.** The pies in Fig. 9 (right) show that at A2, *attn.proj* receives the largest share (32.0%), with *attn.qkv* and *mlp.fc1* close behind. At A3, the distribution becomes more even, but attention still takes the largest share.
- **Depth matters: later blocks require more bits.** The per-block curves (Fig. 10 (top)) increase with depth for both budgets, and the share curves (Fig. 10 (bottom)) peak in the middle-to-late stages, indicating that deeper layers need more precision for stability.
- **A3 mainly lifts the floor.** Upgrading from A2 to A3 shifts the entire per-block curve upward (Fig. 10 (top)), reducing the number of low-precision stretches in both attention and MLP. This suggests a robustness effect: more bits can smooth the activation statistics.

Practical rules-of-thumb. These observations can guide activation mixed precision policies.

- **Prioritize attention first, projection before QKV under tight budgets.** If only a small headroom is available, raise *attn.proj* and then *attn.qkv*.
- **Bias budget to mid/late blocks.** Allocate extra bits to the second half of the network, where attention–attention interactions accumulate and feature distributions widen.

D.4. Visualization Results Comparisons

We present additional visualization results from our DiT-XL/2 model at a 256×256 resolution, using 250 sampling steps (Ho et al., 2020) and a cfg of $= 4.0$ (Ho & Salimans, 2022). Figure 11 compares three activation precision configurations (A4, A3, and A2) across different quantization methods, displaying W1.58 DiT-XL/2 samples for ImageNet (Russakovsky et al., 2015). More visualizations can be found in the supplementary materials.

Table 12. Comparison between uniform and non-uniform quantizers on FLUX-Schnell.

Quantizer	FID ↓	Image Reward ↑	CLIPQA ↑	CLIPScore ↑	PSNR ↑
FP	18.40	0.9323	0.9399	26.54	N/A
Uniform	18.57	0.8617	0.9266	26.34	17.21
Non-Uniform	18.49	0.8698	0.9296	26.41	17.34

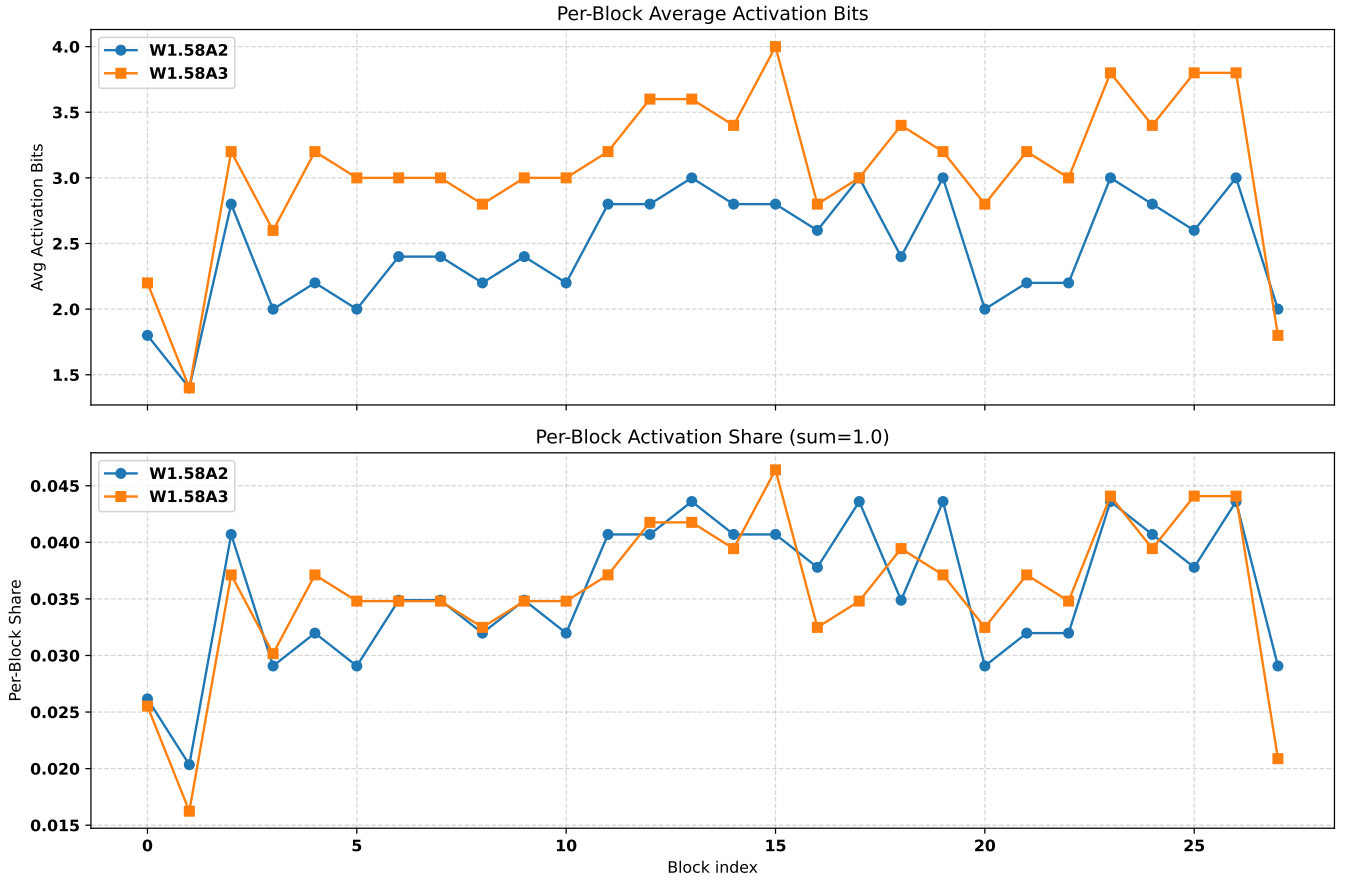
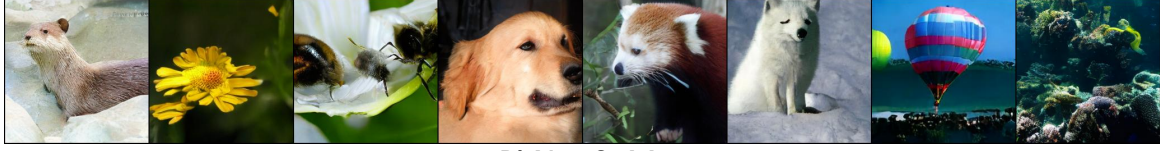


Figure 10. Per-Block Activation Statistics. Top: average activation bits per block; Bottom: normalized per-block share (sums to 1).



BaniryDM-A4



BitNetv2-A4

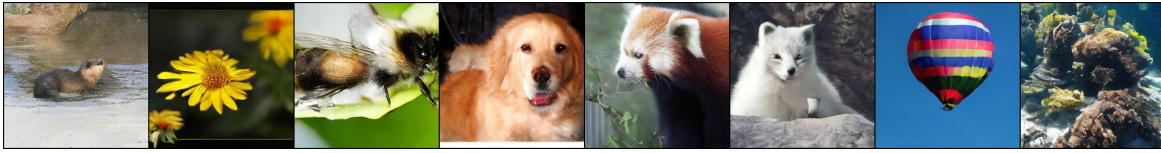


Baseline-A4



RobuQ (w/o AMP)-A4

(a) Activation precision A4.



Baseline-A3



RobuQ-A3



Baseline-A2



RobuQ-A2

(b) Activation precision A2 and A3.

Figure 11. W1.58 DiT-XL/2 samples at 256×256 . Labels = [360, 985, 309, 207, 387, 279, 417, 973]. Cfg = 4.0, sampling steps = 250.